

Guide for Using Scripts 1–3: Abhay/MSK Data Transformation Pipeline

Purpose of this pipeline

This pipeline transforms the raw model-output file sent by Abhay/MSK into a shared, standardized, analysis-ready dataset for our capstone project.

The team request was:

Build generalized R code that can transform the data Abhay sent us so we can do analysis on the same dataset.

The answer is: **Script 1 alone is not enough for the full analysis dataset.**

Script 1 standardizes the raw file, but Scripts 2 and 3 are needed to parse treatment recommendations and create the final analysis-ready dataset.

The full transformation pipeline is:

```
Raw Abhay/MSK CSV
→ Script 1: standardized response-level dataset
→ Script 2: parsed medication-action long dataset
→ Script 3: analysis-ready wide dataset
```

The most important final output for analysis is:

```
data/tranform_data/outputs/analysis_ready_wide.csv
```

Recommended folder structure

Please use the following folder structure:

```
hyperglycemia-ai-capstone/
├─ data/
│   └─ raw/
│       └─ final_results_capstone_data_ver2.csv
│   └─ tranform_data/
│       └─ outputs/
```

```
|— scripts/
|   |— 01_transform_abahy_data_to_standard_format.R
|   |— 02_parse_treatment_plan_to_medication_actions.R
|   |— 03_create_analysis_ready_dataset.R
```

Note: the folder name currently used in the scripts is:

```
data/tranform_data/outputs
```

This is spelled `tranform`, not `transform`. If we want to correct the typo later, we need to update the path in all three scripts.

Required R packages

Before running the scripts, install the required packages:

```
install.packages(c(
  "tidyverse",
  "janitor",
  "stringr",
  "readr",
  "purrr"
))
```

The scripts load the necessary packages internally.

How to run the pipeline

Run the scripts in this exact order:

```
source("scripts/01_transform_abahy_data_to_standard_format.R")
source("scripts/02_parse_treatment_plan_to_medication_actions.R")
source("scripts/03_create_analysis_ready_dataset.R")
```

Do not run Script 2 before Script 1.

Do not run Script 3 before Script 2.

The reason is:

```
Script 1 creates standardized_model_responses.csv
Script 2 needs standardized_model_responses.csv
Script 3 needs both standardized_model_responses.csv and
parsed_medication_actions_long.csv
```

Script 1:

01_transform_abahy_data_to_standard_format.R

Main purpose

Script 1 transforms the raw Abhay/MSK CSV file into a standardized response-level dataset.

It creates a clean backbone where each row represents:

one case × one model × one prompt condition × one run

For the current dataset, this means:

100 cases × 3 models × baseline prompt × run 1 = 300 rows

Input file

Script 1 reads:

data/raw/final_results_capstone_data_ver2.csv

This is the raw file sent by Abhay/MSK.

What Script 1 does

Script 1:

- reads the raw CSV
- cleans column names
- removes empty columns

- standardizes `case_id`
- standardizes model names
- extracts model version information from `model_raw`
- creates `prompt_condition`
- creates `run_number`
- keeps `treatment_plan`
- keeps `vignette_text`
- keeps important clinical variables such as:
 - age
 - sex
 - race
 - HbA1c
 - eGFR
 - BMI
 - CKD
 - heart failure
 - cancer history
 - baseline diabetes medications
- creates a unique `response_id`
- checks for duplicates
- checks case/model coverage
- checks DM026-like case IDs
- saves standardized data and QC files

Main output from Script 1

```
data/transform_data/outputs/standardized_model_responses.csv
```

This is the standardized response-level dataset.

Important columns include:

```
response_id
case_id
model
model_raw
model_version
prompt_condition
run_number
treatment_plan
vignette_text
age
sex
race
```

```
hba1c
egfr
bmi
ckd
heart_failure
cancer_history
baseline_diabetes_drugs
```

QC outputs from Script 1

Script 1 also creates:

```
qc_summary_standardization.csv
duplicate_case_model_prompt_run_rows.csv
dm026_check.csv
case_model_coverage.csv
case_model_coverage_problems.csv
missing_treatment_by_model.csv
dropped_empty_columns.csv
```

`qc_summary_standardization.csv`

This gives a high-level check of the standardized dataset.

It reports:

```
number_of_rows
unique_case_ids
unique_models
unique_prompt_conditions
unique_run_numbers
missing_treatment_plan
duplicated_case_model_prompt_run
case_model_coverage_problems
dm026_like_case_ids_detected
dropped_empty_columns
```

For the current data, the expected result is:

```
300 rows
100 unique cases
3 unique models
```

```
1 prompt condition: baseline
1 run number: 1
0 missing treatment plans
0 duplicate case/model/prompt/run rows
```

`duplicate_case_model_prompt_run_rows.csv`

This file should ideally be empty.

It checks whether the same:

```
case_id × model × prompt_condition × run_number
```

appears more than once.

`dm026_check.csv`

This checks DM026-like IDs, including:

```
DM026
DM026(1)
DM026(2)
DM026(3)
```

This is important because an earlier version of the dataset had a DM026 labeling issue.

`case_model_coverage.csv`

This checks whether every case has responses from all expected models.

For the current dataset, each case should have:

```
Claude Opus
Claude Sonnet
GPT-5.4
```

`case_model_coverage_problems.csv`

This file should ideally be empty.

It lists cases where one or more expected model outputs are missing.

What Script 1 is useful for

Script 1 is useful for:

```
checking dataset structure
checking case IDs
checking model names
checking prompt conditions
checking missing treatment plans
creating a reproducible standardized response-level dataset
```

However, Script 1 does not yet create medication-analysis variables such as:

```
metformin_active
sglt2i_active
glp1_active
insulin_any_active
cost_mentioned
```

Those are created by Scripts 2 and 3.

Script 2:

```
02_parse_treatment_plan_to_medication_actions.R
```

Main purpose

Script 2 parses the `treatment_plan` text into medication-action indicators.

This is necessary because the model outputs are not just numeric variables. The treatment recommendations are embedded in text or table format.

Script 2 tries to distinguish:

```
Medication is actively recommended
Medication is stopped or avoided
Medication is conditional
Medication is mentioned only in rationale
Medication is not preferred
Medication has mixed action, such as switching within the same class
```

This is much better than simple keyword counting.

For example, these two sentences both mention metformin, but they mean different things:

```
Start metformin.  
Avoid metformin because eGFR is below 30.
```

Script 2 is designed to distinguish these cases.

Input file

Script 2 reads:

```
data/tranform_data/outputs/standardized_model_responses.csv
```

This is the output from Script 1.

What Script 2 parses

Script 2 parses the following medication classes:

```
metformin  
SGLT2 inhibitor  
GLP-1 receptor agonist  
dual GIP/GLP-1 agonist  
DPP-4 inhibitor  
any insulin  
basal insulin  
prandial insulin  
sulfonylurea  
TZD / pioglitazone
```

Medication-action variables created by Script 2

For each medication class, Script 2 creates variables such as:

```
mentioned  
active_recommendation
```

```
stop_or_avoid
conditional
caution_monitor
not_preferred
unclear
manual_review_flag
component_mixed_action
source_used
table_has_rows
example_text
```

How Script 2 treats table text vs rationale text

This is an important design feature.

The updated MSK output usually contains a medication table followed by a rationale paragraph.

Script 2 prioritizes the structured medication table.

If a medication appears in the table, the table row is used as the main source for action classification.

If a medication appears only in the rationale paragraph, it is not automatically counted as an active recommendation.

For example:

```
SGLT2 inhibitor is preferred over GLP-1 receptor agonist.
```

This mentions GLP-1, but it does not recommend GLP-1. Script 2 should code GLP-1 as mentioned but not active.

Examples of expected parsing

Example 1

```
Metformin | START
```

Expected coding:

```
metformin_mentioned = 1
metformin_active_recommendation = 1
metformin_stop_or_avoid = 0
```

Example 2

Metformin | STOP due to eGFR below 30

Expected coding:

```
metformin_mentioned = 1
metformin_active_recommendation = 0
metformin_stop_or_avoid = 1
```

Example 3

Dulaglutide | STOP
Semaglutide | START

Expected coding:

```
glp1_active_recommendation = 1
glp1_stop_or_avoid = 1
glp1_component_mixed_action = 1
```

This is not a contradiction. It means the model is switching within the GLP-1 class.

Example 4

Insulin glargine | CONTINUE
Insulin aspart | STOP

Expected coding:

```
insulin_any_active_recommendation = 1
basal_insulin_active_recommendation = 1
prandial_insulin_stop_or_avoid = 1
insulin_any_component_mixed_action = 1
```

This means the patient remains on insulin overall, even though one insulin component is stopped.

Main output from Script 2

```
data/tranform_data/outputs/parsed_medication_actions_long.csv
```

This is a long-format dataset.

Each row represents:

```
one response_id × one drug_class
```

For the current dataset, the expected structure is:

```
300 responses × 10 drug classes = 3000 rows
```

Important columns include:

```
response_id
case_id
model
model_raw
model_version
prompt_condition
run_number
drug_class
mentioned
active_recommendation
start_add
continue
increase_intensify
reduce
stop_or_avoid
conditional
caution_monitor
not_preferred
unclear
manual_review_flag
component_mixed_action
example_text
```

```
source_used  
table_has_rows
```

QC outputs from Script 2

Script 2 also creates:

```
parser_summary_by_drug.csv  
parser_summary_by_source.csv  
parser_manual_review_examples.csv  
parser_mixed_action_examples.csv
```

`parser_summary_by_drug.csv`

Summarizes parser results by drug class.

It reports:

```
n_mentioned  
n_active  
n_stop_or_avoid  
n_conditional  
n_caution_monitor  
n_not_preferred  
n_unclear  
n_manual_review  
n_component_mixed_action
```

This is useful to check whether medication counts look plausible.

`parser_summary_by_source.csv`

Shows whether medication mentions came from:

```
table  
free_text_rationale  
free_text_no_table
```

For the current data, active recommendations should mainly come from `table`, not from `free_text_rationale`.

`parser_manual_review_examples.csv`

Lists rows flagged for manual review.

In the current version, manual review flags mostly come from free-text rationale, not from the structured medication table.

`parser_mixed_action_examples.csv`

Lists within-class mixed actions.

Examples include:

```
stop one GLP-1 drug and start another GLP-1 drug  
continue basal insulin and stop prandial insulin
```

These are not necessarily parser errors.

What Script 2 is useful for

Script 2 is useful for:

```
turning treatment_plan text into structured medication indicators  
distinguishing active recommendation from stop/avoid  
identifying rationale-only mentions  
identifying within-class switches  
flagging ambiguous parser cases for review
```

However, Script 2 is still long-format. For most analysis, we need the wide dataset from Script 3.

Script 3: `03_create_analysis_ready_dataset.R`

Main purpose

Script 3 creates the final analysis-ready wide dataset.

It combines:

```
standardized response-level data from Script 1  
parsed medication-action data from Script 2  
additional text-level indicators
```

The final output has one row per model response and many columns for medication indicators.

This is the file the team should use for most downstream analysis.

Input files

Script 3 reads:

```
data/tranform_data/outputs/standardized_model_responses.csv  
data/tranform_data/outputs/parsed_medication_actions_long.csv
```

Main output from Script 3

```
data/tranform_data/outputs/analysis_ready_wide.csv
```

This is the most important shared analysis dataset.

Each row represents:

```
one model response
```

For the current dataset, it should have:

```
300 rows  
100 cases  
3 models  
baseline prompt condition only  
run_number = 1
```

Important columns in `analysis_ready_wide.csv`

The file includes standardized identifiers:

```
response_id  
case_id  
model  
model_raw  
model_version  
prompt_condition  
run_number
```

It includes the original text:

```
treatment_plan  
vignette_text
```

It includes clinical variables:

```
age  
sex  
race  
hba1c  
egfr  
bmi  
ckd  
heart_failure  
cancer_history  
baseline_diabetes_drugs
```

It includes medication active recommendation indicators:

```
metformin_active  
sglt2i_active  
glp1_active  
gip_glp1_active  
dpp4i_active  
insulin_any_active  
basal_insulin_active  
prandial_insulin_active  
sulfonylurea_active  
tzd_active
```

It includes stop/avoid indicators:

```
metformin_stop_avoid  
sglt2i_stop_avoid  
glp1_stop_avoid  
gip_glp1_stop_avoid  
dpp4i_stop_avoid  
insulin_any_stop_avoid  
basal_insulin_stop_avoid  
prandial_insulin_stop_avoid  
sulfonylurea_stop_avoid  
tzd_stop_avoid
```

It includes conditional and not-preferred indicators:

```
metformin_conditional  
sglt2i_conditional  
glp1_conditional  
metformin_not_preferred  
sglt2i_not_preferred  
glp1_not_preferred
```

It includes mixed-action indicators:

```
component_mixed_action_any  
glp1_component_mixed_action  
insulin_any_component_mixed_action  
basal_insulin_component_mixed_action  
prandial_insulin_component_mixed_action
```

It includes manual review variables:

```
manual_review_any  
manual_review_table  
manual_review_rationale  
manual_review_no_table
```

It includes text-level flags:

```
cost_mentioned  
urgent_eval_mentioned
```

How to interpret key Script 3 variables

`metformin_active`, `sglt2i_active`, `glp1_active`, `insulin_any_active`

These indicate whether the model actively recommended that medication class.

These are the main variables for medication recommendation analysis.

Example use cases:

Compare how often each model recommends SGLT2 inhibitors.
Compare how often each model recommends GLP-1 receptor agonists.
Compare insulin recommendation rates across models.

`stop_avoid` variables

These indicate that the model explicitly stopped, avoided, held, or did not recommend a medication class.

For example:

`metformin_stop_avoid = 1`

may mean the model stopped or avoided metformin, often due to low eGFR.

`not_preferred` variables

These indicate that the medication was mentioned but framed as not preferred, lower priority, not necessary, or not indicated.

This is especially useful for rationale text.

For example:

GLP-1 receptor agonist is not the best fit here.

This should not be counted as active GLP-1 recommendation.

`component_mixed_action_any`

This indicates that a medication class had both active and stop/avoid signals.

This can happen when the model switches within a class.

Examples:

```
Dulaglutide STOP + Semaglutide START  
Insulin glargine CONTINUE + Insulin aspart STOP
```

These are usually clinically meaningful mixed actions, not parser errors.

`manual_review_any`

This means the parser flagged some part of the response for possible review.

However, this can be broad.

For parser reliability, it is better to distinguish:

```
manual_review_table  
manual_review_rationale  
manual_review_no_table
```

`manual_review_table`

This is the most important parser-quality flag.

It indicates possible ambiguity in the structured medication table.

For the current dataset, this should be 0, meaning the table parsing is clean.

`manual_review_rationale`

This indicates ambiguity in rationale-only text.

This is less concerning than table-level ambiguity because rationale-only mentions are not counted as active recommendations.

`cost_mentioned`

This indicates whether the treatment plan mentions cost, affordability, insurance, coverage, or related financial considerations.

This can be used to compare whether models consider cost.

`urgent_eval_mentioned`

This indicates whether the model explicitly recommends urgent evaluation, ED referral, DKA/HHS workup, ketone testing, or hospitalization.

The updated Script 3 uses a strict definition.

It no longer flags generic phrases like:

```
severe hyperglycemia  
no catabolic symptoms  
recent hospitalization
```

For the current dataset, this value may be 0 if models did not explicitly recommend urgent evaluation.

QC outputs from Script 3

Script 3 also creates:

```
script3_summary.csv  
script3_active_rate_by_model.csv  
script3_manual_review_response_list.csv  
script3_urgent_eval_response_list.csv
```

`script3_summary.csv`

This gives the main summary of the analysis-ready dataset.

It reports:

```
number_of_rows  
unique_response_ids  
unique_case_ids  
unique_models
```

```
unique_prompt_conditions
unique_run_numbers
missing_treatment_plan
manual_review_any
manual_review_table
manual_review_rationale
manual_review_no_table
component_mixed_action_any
urgent_eval_mentioned
cost_mentioned
```

For the current dataset, expected values are approximately:

```
number_of_rows = 300
unique_response_ids = 300
unique_case_ids = 100
unique_models = 3
unique_prompt_conditions = 1
unique_run_numbers = 1
missing_treatment_plan = 0
manual_review_table = 0
urgent_eval_mentioned = 0
```

`script3_active_rate_by_model.csv`

This gives preliminary medication recommendation rates by model.

It includes rates for:

```
metformin
SGLT2 inhibitor
GLP-1 receptor agonist
GIP/GLP-1 agonist
DPP-4 inhibitor
insulin
basal insulin
prandial insulin
sulfonylurea
TZD
cost mentioned
urgent evaluation mentioned
manual review flags
```

This is useful for early descriptive analysis and team discussion.

```
script3_manual_review_response_list.csv
```

This lists responses with parser manual-review flags.

In the current output, the flags are rationale-level rather than table-level.

Use this file if we want to inspect ambiguous rationale text.

```
script3_urgent_eval_response_list.csv
```

This lists responses where `urgent_eval_mentioned = 1`.

For the current dataset, this file may be empty because the stricter urgent-evaluation definition removed false positives.

Which output should teammates use?

For main analysis

Use:

```
data/tranform_data/outputs/analysis_ready_wide.csv
```

This is the main shared dataset.

For checking raw standardized responses

Use:

```
data/tranform_data/outputs/standardized_model_responses.csv
```

This is useful if someone wants to inspect the original treatment plan text with standardized identifiers.

For parser validation

Use:

```
data/tranform_data/outputs/parsed_medication_actions_long.csv  
parser_manual_review_examples.csv  
parser_mixed_action_examples.csv
```

These are useful for checking whether medication parsing is working correctly.

For preliminary model comparison

Use:

```
data/tranform_data/outputs/script3_active_rate_by_model.csv
```

This gives quick model-level medication recommendation rates.

For QC

Use:

```
qc_summary_standardization.csv  
case_model_coverage.csv  
case_model_coverage_problems.csv  
parser_summary_by_drug.csv  
parser_summary_by_source.csv  
script3_summary.csv
```

Current dataset interpretation

The current dataset appears to be:

```
baseline prompt only  
one run per case/model  
100 unique cases
```

```
3 models
300 total responses
```

Therefore:

```
prompt_condition = baseline
run_number = 1
```

This does not mean repeated-run analysis is available. It only creates a structure that can support repeated runs in the future.

Because this dataset does not yet include ADA-guided outputs, we should not run baseline vs ADA-guided comparisons yet.

Future ADA-guided data

When ADA-guided outputs are available, they should be transformed into the same structure.

The key column will be:

```
prompt_condition
```

Possible values may include:

```
baseline
ada_guided
```

Once both are available, we can compare recommendations within the same:

```
case_id × model
```

across prompt conditions.

Future repeated-run data

For intra-model stability analysis, we need repeated runs.

The key column will be:

```
run_number
```

For example:

```
DM001, GPT-5.4, baseline, run 1  
DM001, GPT-5.4, baseline, run 2  
DM001, GPT-5.4, baseline, run 3  
DM001, GPT-5.4, baseline, run 4  
DM001, GPT-5.4, baseline, run 5
```

The current dataset has only:

```
run_number = 1
```

So repeated-run stability should not be interpreted from this file.

Important notes and limitations

This pipeline is designed to be reproducible and generalized, but the medication parser is still rule-based.

Therefore:

```
The parsed indicators should be treated as structured approximations of the  
model output.  
The team should manually validate a sample before final manuscript analysis.
```

Recommended validation:

```
10 random table-based active recommendations  
10 rationale-only mentions  
all mixed-action examples  
10 manual-review examples
```

The current parser is strongest for structured medication table outputs. It is less definitive for nuanced rationale-only text.

Summary

Script 1 is necessary but not sufficient for the full analysis dataset.

The three-script pipeline should be used together:

Script	Purpose	Main output
Script 1	Standardize Abhay/MSK raw data	<code>standardized_model_responses.csv</code>
Script 2	Parse medication actions from treatment plans	<code>parsed_medication_actions_long.csv</code>
Script 3	Create final analysis-ready wide dataset	<code>analysis_ready_wide.csv</code>

The main file for team analysis is:

```
data/tranform_data/outputs/analysis_ready_wide.csv
```

This file should be used for preliminary descriptive analysis, model comparison, medication recommendation rates, and future formal analyses.